

20-Advanced Python Functions, Namespaces, and Lambda

Introduction

- Continuation of a previous class on Python functions.
- Today's topics: built-in functions, anonymous (lambda) functions, filter, map, and reduce, plus concepts of function referencing, aliasing, namespaces, and scope.

Function Referencing and Aliasing

- In Python, defining a function creates both a name (identifier) and a function object in memory.
- You can assign a function to another variable (aliasing); both names then reference the same function object (same memory address).
- Function references can be printed (displays function's address), and without parentheses, Python provides information about the function rather than calling it.
- Aliasing demonstrates that both variables (aliases) point to the same object.

Namespaces and Scope

- **Namespace:** Container holding names (identifiers) and references.
 - Types of namespaces:
 - **Built-in namespace:** Loaded by default with the standard Python library; contains functions like `print`, `input`, `int`, `str`, `float`, `abs`, `min`, `max`, etc.

- **Global namespace:**

Anything defined outside functions is global; code written at the outermost level.

- **Local namespace:**

Names defined within a function's indented block; only exists during function execution.

- **Scope:** Rule that determines how Python resolves variable names. The order is:

1. Local namespace

2. Global namespace

3. Built-in namespace

- If Python doesn't find the name, it raises a `NameError`.

- Code samples illustrate how variables are resolved and the error locations if names are not defined in the correct scope.

Example Walkthrough (Code and Memory)

- Step-by-step analysis of a code block with nested functions, showing variable assignments and how each namespace is created and accessed, both during and after function calls.

- Demonstrates:

- Local vs. global variable resolution

- Memory for function objects and deletion of local namespace after function completion

- Error generation when accessing undefined or out-of-scope variables

- Clarifies that objects are stored in Python's heap, while namespaces just track references (names) to those objects.

Closures/Returning Function References

- Shows how a function can return a reference to its inner function (closure).
- When a function returns a reference (not a call) of its defined inner function, this can then be called from the outer/global scope.
- Examples show how function references are stored and how closures work in practice.
- Closure concept is a precursor to decorators in Python.

Anonymous (Lambda) Functions

- **Anonymous functions:** Functions without a name, created using `lambda`.
- Syntax: `lambda arguments: expression`
- Lambda functions are inline, can have multiple arguments, and always return the value of the single expression provided.
- Typically used for quick, throwaway operations where a function object is only required temporarily.
- Lambda can be stored in a variable for later use, but often used as an argument to functions.

Lambda in Practice

- Example: `lambda num: num ** 2` creates a function that squares its input.

- Lambda can be invoked directly or assigned and used by reference.
- Discussion around inline applications (e.g., with filter, map, sort functions).
- Students are asked to practice by writing a lambda function that extracts a substring (exercise suggestion).

Q&A and Assignments

- Clarification on lambda parameter binding and scoping.
- Explanation about returning multiple values from functions using tuple unpacking.
- Future assignments teased: set and test assignments, functional programming assignment announced.

Key Takeaways

- Understanding the distinctions and interplay between function references, namespaces, and scoping rules in Python is essential for effective programming.
- Lambda functions provide concise, inline function creation, heavily used in functional programming paradigms.
- Closures and returning function references are foundational for more advanced patterns like decorators.
- Comprehensive in-class discussion ensures students understand both theory and Python's real memory handling logic.